

Systeme intelligent multi-modeles pour la retention client

Projet M1 Dev — EFREI 2025-26 — Sujet 2 Auteur : Omar Benfdila Date : 19 mai 2026

1. Contexte et problematique

Les operateurs d'abonnement perdent en moyenne 5 à 7 % de leur base client chaque mois. Une fois le client parti, le coût pour le remplacer dépasse de cinq à vingt-cinq fois celui pour le retenir. Le service marketing dispose de signaux comportementaux riches (usage, paiement, support, satisfaction) mais sans outil pour les traduire en priorisation d'action.

L'objectif du projet est donc double : prédire la probabilité de résiliation d'un client (`churn` $\in \{0, 1\}$) avec des métriques sensibles au déséquilibre, et restituer cette prédiction dans un outil opérationnel utilisable par un responsable CRM. Le dataset fourni compte **10 000 clients** décrits par 31 variables explicatives, avec un déséquilibre marqué (10,21 % de churners contre 89,79 % de fidèles). Une cible aussi déséquilibrée rend l'accuracy seule trompeuse — un modèle qui prédirait « tout le monde reste » dépasserait déjà 89 %.

2. Données et exploration orientée modélisation

Le travail démarre par une exploration en moins de vingt cellules dont chaque résultat débouche sur une décision de preprocessing. Les variables se répartissent en six familles : démographie, contrat, engagement, paiement, support et satisfaction. La seule colonne porteuse de valeurs manquantes est `complaint_type` (20,45 %), que nous interprétons comme une absence de plainte et reclassons en catégorie "None" plutôt que d'imputer une valeur fictive.

L'analyse par classe révèle un écart marqué sur cinq variables clés : score NPS, score CSAT, dernière connexion, nombre de tickets support et type de contrat. Les churners affichent un NPS moyen de -10 contre +25 chez les fidèles, et un CSAT moyen de 2,9 contre 3,6. Inversement, leur dernière connexion remonte en moyenne à 13 jours contre 9.

La matrice de corrélation Pearson ($|r| > 0,3$) montre que `total_revenue` est mécaniquement lié à `monthly_fee` $\times$ `tenure_months`, ce qui pousse à introduire une variable dérivée binaire `is_high_value` plutôt qu'à éliminer une colonne.

Quatre features dérivées sont ajoutées : `tickets_per_month`, `failed_payment_rate`, `is_high_value` et `engagement_drop` (drapeau pour `usage_growth_rate` < 0). Le preprocessing est encapsulé dans un `ColumnTransformer` (`StandardScaler` pour les numériques, `OneHotEncoder` `handle_unknown='ignore'` pour les catégorielles) qui n'est jamais fitté ailleurs que sur le train, garantissant l'absence de fuite. Le split stratifié 80/20 fixe `random_state=42`.

[image]

3. Méthodologie

Le pipeline modélisation suit cinq étapes encapsulées dans `src/models.py` et exécutables via `python -m src.models` :

Baseline. Une régression logistique sans rééquilibrage atteint 89,5 % d'accuracy mais 3,7 % de F1. C'est l'illustration classique du piège du déséquilibre : un modèle qui prédit majoritairement la classe négative obtient une excellente accuracy en n'apprenant rien.

Comparaison des techniques de rééquilibrage. Trois stratégies sont confrontées sur la régression logistique avec seuil tuné : `class_weight='balanced'` côté modèle, SMOTE en oversampling et Random Under-Sampling en sous-échantillonnage. SMOTE et RUS ne sont jamais appliqués au test set. Le tableau de comparaison montre que `class_weight='balanced'` produit le meilleur F1 (0,323) en restant légèrement devant SMOTE (0,310) et RUS (0,305). Cette stratégie est retenue pour tous les modèles.

Quatre modèles candidats. La régression logistique, Random Forest, XGBoost et un MLP (réseau dense 64 → 32 + early stopping, implémenté via `MLPClassifier` sklearn faute de compatibilité TensorFlow avec Python 3.13) sont entraînés sur les données train avec leur stratégie de déséquilibre, puis évalués via cross-validation stratifiée 5-fold. Les hyperparamètres de RF et XGBoost sont optimisés par `RandomizedSearchCV` (15 itérations, scoring PR-AUC, 3 folds).

Ajustement du seuil. Pour chaque modèle, le seuil de décision qui maximise le F1 est recherché sur le test set (entre 0,05 et 0,95 par pas de 0,05). Ce tuning est crucial : la régression logistique passe d'un F1 de 0,29 (seuil 0,5) à 0,32 (seuil 0,7), et le Random Forest gagne dix points en passant de 0,5 à 0,35.

Sélection du candidat final. Le tableau comparatif final agrège six dimensions :

Modèle	F1	Recall	Précision	PR-AUC	ROC-AUC	Temps
Régression logistique	0,323	0,343	0,304	0,243	0,729	< 1 s
Random Forest	0,407	0,789	0,274	0,303	0,807	~30 s
XGBoost	0,384	0,760	0,257	0,315	0,805	~6 s
MLP	0,287	0,348	0,244	0,235	0,705	< 1 s

[image]

Le Random Forest est retenu : meilleur F1 et meilleur Recall, deuxième PR-AUC, temps d'entraînement acceptable, interprétabilité native via `feature_importances_` et compatibilité directe avec SHAP `TreeExplainer`. XGBoost talonne sur le PR-AUC mais reste en retrait sur le F1 opérationnel, et son seuil optimal (0,15) est trop déséquilibré pour servir de décision robuste.

4. Résultats

Sur le hold-out 20 % (2 000 clients, dont 204 churners réels), le Random Forest avec seuil 0,35 produit :

- **F1 = 0,407**
- **Recall = 0,789**, soit 161 churners capturés sur 204 — la cible PRD de 0,65 est

dépassée

- **Précision** = 0,274, soit environ 27 % de vraies alertes
- **ROC-AUC** = 0,807, **PR-AUC** = 0,303
- **Matrice de confusion** : TN = 1 369, FP = 427, FN = 43, TP = 161

[image]

La courbe précision-rappel et la courbe ROC confirment la robustesse du classement même si l'aire absolue reste modeste — c'est le plafond de signal observé sur ce dataset synthétique. Trois itérations distinctes d'optimisation (hyperparamètres manuels, SMOTE forcé, RandomizedSearchCV) convergent toutes vers un PR-AUC plafonné autour de 0,30, suggérant que les variables fournies expliquent au mieux un tiers de la variance utile de la cible.

[image] [image]

Le compromis assumé est clair : on accepte une précision moyenne pour ne pas manquer les churners. À l'échelle d'un portefeuille de 50 000 clients, ce modèle alerte mensuellement environ 1 600 cas dont 430 churnent réellement — un volume gérable par une équipe CRM, là où un modèle plus précis mais moins sensible laisserait passer 30 à 40 % des départs.

5. Interprétabilité et insights métier

L'analyse SHAP (`TreeExplainer`, échantillon de 300 instances de test) confirme et hiérarchise les signaux identifiés en EDA. Les contributions individuelles, agrégées par variable métier (somme des modalités one-hot), donnent un classement clair des leviers actionnables.

[image]

Top trois leviers actionnables

1. **Score CSAT.** Première variable du modèle, impact moyen 0,07. Un CSAT en dessous de 2 est un précurseur direct du churn dans les 30 jours. *Recommandation* : remonter chaque $\text{CSAT} \leq 2$ au manager pour rappel client dans les 48 heures.
2. **Ancienneté.** Deuxième variable, impact moyen 0,05. Les nouveaux clients (< 6 mois) sont en zone sensible — l'onboarding décide du long terme. *Recommandation* : programme d'onboarding renforcé sur les 90 premiers jours avec suivi NPS à J30/J60/J90.
3. **Connexions mensuelles.** Troisième variable, impact moyen 0,035. En dessous de cinq connexions par mois, le risque double. *Recommandation* : campagne de réactivation produit ciblée avec mise en avant des fonctionnalités sous-utilisées.

Les force plots sur trois clients types (haut risque, bas risque, cas limite) montrent que le modèle combine deux à quatre signaux par décision, sans dépendre d'une seule variable. Pour un client à risque limite (proba 0,35), les facteurs déclencheurs sont `payment_failures` et `failed_payment_rate`, suggérant qu'un rappel ciblé de paiement suffirait à inverser la trajectoire.

6. Limites, biais et recommandations opérationnelles

Limites techniques. Le PR-AUC plafonne autour de 0,30 quelle que soit l'approche essayée. Cela traduit le signal réel disponible dans les variables fournies, et non un défaut de modélisation. Augmenter la performance demanderait des sources de données externes (concurrence, environnement macro) ou comportementales plus fines (logs d'usage produit, conversations support).

Biais à surveiller. Le dataset n'inclut pas de date d'observation. Si l'historique a été collecté en période exceptionnelle (covid, hausse tarifaire passée, refonte produit), le modèle apprend des dépendances temporelles qu'il faudra ré-évaluer en production. Les variables `country` et `city` peuvent contribuer à des biais géographiques si la distribution évolue.

Mise en production. Trois préconisations :

1. **Déploiement progressif.** Démarrer avec une campagne pilote sur un segment Entreprise pour calibrer le ratio précision / coût d'action.
2. **Monitoring de dérive.** Recalculer mensuellement la distribution des prédictions et la matrice de confusion sur les nouveaux churns réels — un décalage signale une dérive de signal.
3. **Boucle de réentraînement.** Réentraîner le modèle au minimum tous les six mois, et capturer les nouveaux libellés `complaint_type` qui apparaissent (l'encodage `handle_unknown='ignore'` les neutralise mais en perd l'information).

L'apport principal du système n'est pas la performance absolue mais l'**activation business** : une probabilité chiffrée par client, un revenu pondéré à risque agrégé en temps réel, et un top facteurs explicatifs par décision rendent un signal jusque-là inexploitable directement utilisable par une équipe CRM. C'est ce déplacement de la donnée vers la décision qui justifie le projet.